

LAPORAN SISTEM OPERASI

TUGAS 1 – 4



Dosen Pembimbing:

Triawan Adi Cahyanto M.Kom

Dibuat oleh :

Nadiva Nathania (2410651024)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025/2026

TUGAS 1 PROCESS MANAGER

Buat program sederhana yang mengelola process menggunakan konsep forking dan inter process communication (IPC) dasar. Program ini harus mendemonstrasikan pemahaman tentang process creation, synchronization, dan komunikasi.

```
nadiva_nathania@LAPTOP-JR1G744R:~$ mkdir -p praktikum_os/monitor_dir
cd praktikum_os/monitor_dir
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ nano process_manager.c
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ gcc process_manager.c -o process_manager
process_manager.c: In function 'find_primes':
process_manager.c:28:30: warning: ' ' directive writing 1 byte into a region of size between 0 and 9 [-Wformat-overflow]
    28 |         sprintf(temp, "%d ", i);
        |                      ^
process_manager.c:28:13: note: 'sprintf' output between 3 and 12 bytes into a destination of size 10
    28 |         sprintf(temp, "%d ", i);
        |         ^
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ ./process_manager
=== Tugas 1: Process Manager (NIM: 4, varian GENAP / Shared Memory) ===
[Child 1] Mulai...
[Child 1] Masukkan angka untuk faktorial: 5
[Child 2] Mulai...
[Child 2] Masukkan batas bilangan prima: 10
[Child 3] Mulai...
[Child 3] Masukkan string: 23
[Hasil Child 1] Faktorial: 120
[Child 1] Selesai. Waktu: 2.00 detik

[Parent] Child selesai. PID=487 | Exit=0 | Time=0.000704 detik
[Hasil Child 2] Bilangan Prima: 2 3 5 7
[Child 2] Selesai. Waktu: 3.00 detik

[Parent] Child selesai. PID=488 | Exit=0 | Time=0.000971 detik
[Hasil Child 3] String Terbalik: 32
[Child 3] Selesai. Waktu: 4.00 detik

[Parent] Child selesai. PID=489 | Exit=0 | Time=0.001146 detik

=== Semua proses selesai ===
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ nano process_manager.c
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$
```

Kode file process_manager.c :

```
GNU nano 7.2 process_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

// --- Fungsi Tugas ---
// Child 1: Faktorial
long long factorial(int n) {
    if (n == 0) return 1;
    return (long long)n * factorial(n - 1);
}

// Child 2: Bilangan Prima (sederhana)
void find_primes(int limit, char *output) {
    strcpy(output, "");
    for (int i = 2; i <= limit; i++) {
        int is_prime = 1;
        // ... (prime check logic)
        if (is_prime) {
            // PERBAIKAN: Ukuran buffer ditingkatkan untuk menyimpan angka 10 digit + spasi + terminator null dengan aman.
            // 15 byte adalah angka bulat yang aman untuk 'int' standar guna mencegah luapan.
            char temp[15];
            sprintf(temp, "%d ", i);
            strcat(output, temp);
        }
    }
}

// Child 3: String Terbalik
void reverse_string(const char *input, char *output) {
    int len = strlen(input);
```

```

GNU nano 7.2 process_manager.c
    for (int i = 0; i < len; i++) {
        output[i] = input[len - 1 - i];
    }
    output[len] = '\0';
}

// --- Struktur Data untuk Shared Memory Sederhana (digantikan oleh file/pipe di contoh ini untuk kesederhanaan) ---
// Dalam implementasi nyata, data ini akan ada di Shared Memory (shmget/shmat).

void run_child(int child_id, int input_val) {
    time_t start_time = time(NULL);
    printf("[Child %d] Mulai...\n", child_id);

    if (child_id == 1) {
        // Simulasi input: Angka untuk Faktorial
        printf("[Child 1] Masukkan angka untuk faktorial: %d\n", 5);
        long long result = factorial(5);
        // sleep() digunakan untuk menunda eksekusi (simulasi waktu pemrosesan yang lama)
        sleep(2);
        printf("[Hasil Child 1] Faktorial: %lld\n", result);
    } else if (child_id == 2) {
        // Simulasi input: Batas Bilangan Prima
        printf("[Child 2] Masukkan batas bilangan prima: %d\n", 10);
        char result[20];
        find_primes(10, result);
        // sleep() digunakan untuk menunda eksekusi (simulasi waktu pemrosesan yang lama)
        sleep(3);
        printf("[Hasil Child 2] Bilangan Prima: %s\n", result);
    } else if (child_id == 3) {
        // Simulasi input: String
        const char *input_str = "23";
        printf("[Child 3] Masukkan string: %s\n", input_str);
        char result[5];

```

```

GNU nano 7.2 process_manager.c
        reverse_string(input_str, result);
        // sleep() digunakan untuk menunda eksekusi (simulasi waktu pemrosesan yang lama)
        sleep(4);
        printf("[Hasil Child 3] String Terbalik: %s\n", result);
    }

    time_t end_time = time(NULL);
    printf("[Child %d] Selesai. Waktu: %.2f detik\n", child_id, difftime(end_time, start_time));
    // exit(0) digunakan untuk menghentikan proses *child* dan mengembalikan status 0 ke *parent*.
    // Ini penting agar *parent* tahu proses *child* sudah selesai.
    exit(0);
}

int main() {
    printf("=== Tugas 1: Process Manager (NIM: 8, varian GENAP / Shared Memory) ===\n");
    pid_t pids[3];
    int status;
    clock_t start_time, end_time;

    // --- Membuat dan Menjalankan Child 1 ---
    start_time = clock();
    // fork() digunakan untuk membuat proses baru (Child 1) yang merupakan duplikat dari proses saat ini (Parent).
    pids[0] = fork();
    if (pids[0] == 0) { run_child(1, 5); } // Ini kode untuk Child.
    // perror() dan exit() digunakan untuk melaporkan dan menghentikan program jika fork gagal.
    else if (pids[0] < 0) { perror("fork 1 error"); exit(1); }

    // --- Membuat dan Menjalankan Child 2 ---
    // fork() digunakan untuk membuat proses baru (Child 2). Sekarang ada tiga proses: Parent, Child 1, dan Child 2.
    pids[1] = fork();
    if (pids[1] == 0) { run_child(2, 10); } // Ini kode untuk Child.
    else if (pids[1] < 0) { perror("fork 2 error"); exit(1); }
}

```

```

// --- Parent Menunggu dan Melaporkan Hasil (Berurutan) ---
for (int i = 0; i < 3; i++) {
    // waitpid() digunakan oleh *parent* untuk menanggukuhkan eksekusinya dan menunggu proses *child*
    // tertentu (ditentukan oleh pids[i]) selesai. Ini memastikan *parent* tidak keluar
    // sebelum *child* menyelesaikan tugasnya (mencegah *zombie process*).
    pid_t child_pid = waitpid(pids[i], &status, 0);
    end_time = clock();
    double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;

    printf("\n[Parent] Child selesai. PID=%d | Exit=%d | Time=%.6f detik\n",
        child_pid, WEXITSTATUS(status), time_taken);
    // Dalam implementasi sebenarnya, di sini Parent akan membaca data dari Shared Memory
    // Untuk contoh ini, output hasil ada di dalam fungsi Child.
}

printf("\n=== Semua proses selesai ===\n");
return 0;
}

```

Laporan:

- system call fork () ini digunakan untuk membuat proses baru karena fork memungkinkan parent process untuk membagi dirinya menjadi proses identik, sehingga menjalankan fungsi yang berbeda.

- Parent process harus menggunakan waitpid() untuk menunggu setiap child selesai dieksekusi, sebelum parent melanjutkan atau keluar, perlu untuk mengumpulkan status keluar child dan mencegah child process menjadi zombie.
- Setelah itu tampilkan PID child, status exit dengan menggunakan WIFEXITED dan WEXITSTATUS.
- WIFEXITED digunakan untuk memeriksa apakah child proses telah berakhir secara normal.
Cara kerjanya menerima nilai integer status yang dikembalikan oleh wait() atau waitpid().
- WEXITSTATUS digunakan untuk mendapatkan kode keluar yang dikirimkan oleh child process.
Cara kerjanya menerima nilai integer status yang dikembalikan oleh wait() atau waitpid().
- system call pipe() ini digunakan untuk komunikasi, dan setiap child mengirim hasil komputasinya ke parent melalui pipe unik per child. Parent membaca dari pipe dan menampilkan hasil.
- Variasi unik sesuai digit NIM terakhir saya genap yaitu [4] tambahkan signal handling di parent untuk menangani SIGINT (Ctrl+C) agar gracefully terminate semua child.
- Signal Handling ini digunakan untuk sebuah program reaksi secara terprogram.

```

process_manager.c: In function 'main':
process_manager.c:113:1: error: stray '\302' in program
113 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:113:2: error: stray '\302' in program
113 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:113:3: error: stray '\302' in program
113 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:113:4: error: stray '\302' in program
113 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:113:11: error: stray '\302' in program
113 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~

```

```

process_manager.c: In function 'main':
process_manager.c:116:1: error: stray '\302' in program
116 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:116:2: error: stray '\302' in program
116 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:116:3: error: stray '\302' in program
116 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:116:4: error: stray '\302' in program
116 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~
process_manager.c:116:11: error: stray '\302' in program
116 | <U+00A0><U+00A0><U+00A0><U+00A0>return<U+00A0>0;
    | ^~~~~~

```

- Konsep zombie process dan bagaimana dihindari, serta catat minimal 2 error yang ditemui selama debugging.
- Zombie process adalah proses yang telah selesai dieksekusi.

Cara menghindarinya:

- Menggunakan wait() atau waitpid()
 - Menggunakan Signal Handler
 - Double Fork
- Dokumentasi di atas adalah error yang terjadi saat error stray '\302' in program pada baris kode 113 dan 116 dari file process_manager.c kesalahan ini merupakan kompilasi yang terjadi ketika menggunakan karakter yang tidak dikenali.
- Solusinya dengan memperbaiki kesalahan pada baris 113 dan 116, yaitu dengan menghapus dan mengetik ulang karakter-karakter yang bermasalah.

Caranya:

- Buka kembali file process_manager.c menggunakan editor yang sama yaitu nano.
 - Lalu pergi ke baris 113 dan 116 pada baris yang bermasalah, hapus seluruh baris yang ditunjukkan 113 dan 116.
 - Setelah itu ketik ulang baris tersebut dari awal, pastikan menggunakan tombol spasi biasa dan tanda baca keyboard standar.

TUGAS 2 FILE MONITOR

Buat program yang memantau perubahan pada direktori tertentu dan mencatat aktivitas file, menggunakan system calls untuk file operations.

```
nadiva_nathania@LAPTOP-JR1G744R:~$ mkdir -p praktikum_os/monitor_dir
cd praktikum_os/monitor_dir
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ nano directory_monitor.c
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ mkdir -p /tmp/monitor_dir
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ gcc directory_monitor.c -o monitor
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ ./monitor /tmp/monitor_dir

--- Pemantau File System Dimulai ---
Direktori yang dipantau: /tmp/monitor_dir
Log akan dicatat ke: file_activity.log
Filter aktif: Hanya .txt dan .log yang dicatat (NIM Genap).
Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.

[2025-10-10 21:43:50] CREATED      data.txt                [PID: 523]
[2025-10-10 21:44:01] CREATED      server.log             [PID: 523]
[2025-10-10 21:44:01] MODIFIED     server.log             [PID: 523]
[2025-10-10 21:44:31] DELETED      data.txt                [PID: 523]
[2025-10-10 21:44:45] CREATED      data.txt                [PID: 523]
[2025-10-10 21:44:45] MODIFIED     data.txt                [PID: 523]
^C

Sinyal 2 diterima. Melakukan pembersihan...
Watch descriptor dihapus.
Inotify file descriptor ditutup.
Log file descriptor ditutup: file_activity.log
Program pemantauan dihentikan dengan bersih.
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ cat file_activity.log
[2025-10-10 21:43:50] CREATED      data.txt                [PID: 523]
[2025-10-10 21:44:01] CREATED      server.log             [PID: 523]
[2025-10-10 21:44:01] MODIFIED     server.log             [PID: 523]
[2025-10-10 21:44:31] DELETED      data.txt                [PID: 523]
[2025-10-10 21:44:45] CREATED      data.txt                [PID: 523]
[2025-10-10 21:44:45] MODIFIED     data.txt                [PID: 523]
nadiva_nathania@LAPTOP-JR1G744R:~/praktikum_os/monitor_dir$ |
```

Pada terminal ke 2 :

```
nadiva_nathania@LAPTOP-JR1G744R:~$ touch /tmp/monitor_dir/data.txt
nadiva_nathania@LAPTOP-JR1G744R:~$ echo "halo" >> /tmp/monitor_dir/server.log
nadiva_nathania@LAPTOP-JR1G744R:~$ touch /tmp/monitor_dir/config.json
nadiva_nathania@LAPTOP-JR1G744R:~$ rm /tmp/monitor_dir/data.txt
nadiva_nathania@LAPTOP-JR1G744R:~$ echo "test" > /tmp/monitor_dir/data.txt
nadiva_nathania@LAPTOP-JR1G744R:~$
```

Kode file directory_monitor.c :

```
GNU nano 7.2 directory_monitor.c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <signal.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <fcntl.h> // Diperlukan untuk open()

// --- KONSTANTA ---
// Ukuran perkiraan untuk satu event (termasuk nama file, biasanya 16 byte)
#define EVENT_SIZE (sizeof(struct inotify_event))
// Ukuran buffer untuk menampung 1024 event secara berurutan
#define EVENT_BUF_LEN (1024 * (EVENT_SIZE + 16))
#define LOG_FILE_NAME "file_activity.log"
#define MONITOR_PATH "/tmp/monitor_dir" // Direktori default yang akan dipantau

// --- VARIABEL GLOBAL UNTUK CLEANUP ---
// File descriptor untuk instance inotify
int inotify_fd = -1;
// File descriptor untuk file log
int log_fd = -1;
// Watch descriptor untuk direktori yang dipantau
int watch_descriptor = -1;

/**
 * @brief Memeriksa apakah nama file memiliki ekstensi .txt atau .log.
 * * Ini adalah implementasi filter untuk NIM genap.
 * * @param filename Nama file yang akan diperiksa.
 * * @return 1 jika relevan (.txt atau .log), 0 jika tidak.
 */
```

```

GNU nano 7.2 directory_monitor.c
int is_relevant_file(const char *filename) {
    if (filename == NULL || filename[0] == '\0') {
        return 0; // Abaikan direktori atau event tanpa nama
    }
    size_t len = strlen(filename);
    // Pastikan panjang cukup untuk menampung ekstensi minimum (misal: ".txt")
    if (len >= 4 && (strcmp(filename + len - 4, ".txt") == 0)) {
        return 1;
    }
    if (len >= 4 && (strcmp(filename + len - 4, ".log") == 0)) {
        return 1;
    }
    return 0;
}

/**
 * @brief Mendapatkan timestamp dan memformatnya ke string.
 * @param buffer Buffer untuk menyimpan string timestamp yang diformat.
 * @param buf_size Ukuran buffer.
 */
void get_formatted_timestamp(char *buffer, size_t buf_size) {
    time_t t;
    struct tm *local_time;

    // 1. Ambil waktu saat ini (time())
    t = time(NULL);
    // 2. Konversi ke waktu lokal (localtime())
    local_time = localtime(&t);

    // Format string: [YYYY-MM-DD HH:MM:SS]
    strftime(buffer, buf_size, "[%Y-%m-%d %H:%M:%S]", local_time);
}

```

```

/**
 * @brief Handler sinyal (SIGINT, SIGTERM) untuk membersihkan resource.
 */
void cleanup_handler(int sig) {
    printf("\n\nSinyal %d diterima. Melakukan pembersihan...\n", sig);

    if (watch_descriptor != -1 && inotify_fd != -1) {
        // Hapus pemantauan
        inotify_rm_watch(inotify_fd, watch_descriptor);
        printf("Watch descriptor dihapus.\n");
    }

    if (inotify_fd != -1) {
        // Tutup file descriptor inotify
        close(inotify_fd);
        printf("Inotify file descriptor ditutup.\n");
    }

    if (log_fd != -1) {
        // Tutup file descriptor log
        close(log_fd);
        printf("Log file descriptor ditutup: %s\n", LOG_FILE_NAME);
    }

    printf("Program pemantauan dihentikan dengan bersih.\n");
    exit(EXIT_SUCCESS);
}

/**
 * @brief Fungsi utama program pemantauan direktori.
 */
int main(int argc, char **argv) {
    char buffer[EVENT_BUF_LEN];

```

```

GNU nano 7.2 directory_monitor.c
int length;
char *dir_to_monitor = MONITOR_PATH;

// Jika argumen diberikan, gunakan sebagai direktori yang dipantau
if (argc > 1) {
    dir_to_monitor = argv[1];
}

printf("--- Pemantau File System Dimulai ---\n");
printf("Direktori yang dipantau: %s\n", dir_to_monitor);
printf("Log akan dicatat ke: %s\n", LOG_FILE_NAME);
printf("Filter aktif: Hanya .txt dan .log yang dicatat (NIM Genap).\n");
printf("Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.\n\n");

// 1. Setup Signal Handler
// Untuk menghentikan loop dengan bersih saat Ctrl+C (SIGINT) atau SIGTERM
signal(SIGINT, cleanup_handler);
signal(SIGTERM, cleanup_handler);

// 2. Inisialisasi inotify
// Menggunakan inotify_init() untuk mendapatkan file descriptor baru
inotify_fd = inotify_init();
if (inotify_fd < 0) {
    perror("ERROR: inotify_init");
    return EXIT_FAILURE;
}

// 3. Buka File Log
// Menggunakan open() dengan O_CREAT (buat jika belum ada), O_WRONLY (mode tulis), O_APPEND (tambahkan ke akhir)
// 0644 adalah permission: rw-r--r--
log_fd = open(LOG_FILE_NAME, O_CREAT | O_WRONLY | O_APPEND, 0644);

```



```

GNU nano 7.2 directory_monitor.c
        continue; // Lanjutkan loop jika diinterupsi oleh sinyal
    }
    perror("ERROR: read inotify buffer");
    cleanup_handler(0);
    return EXIT_FAILURE;
}

// --- MEMPROSES BUFFER EVENT ---
int i = 0;
// Kita membaca buffer event yang mungkin berisi banyak event sekaligus
while (i < length) {
    // CAST: Ambil struktur inotify_event dari posisi 'i' dalam buffer
    struct inotify_event *event = (struct inotify_event *) &buffer[i];

    // Variabel untuk informasi log
    char timestamp_str[30];
    char log_message[256];
    const char *event_type_str = "UNKNOWN";

    // Dapatkan Timestamp
    get_formatted_timestamp(timestamp_str, sizeof(timestamp_str));

    // Jika event memiliki nama (nama != NULL) DAN bukan direktori
    if (event->len && !(event->mask & IN_ISDIR)) {
        // Pengecekan Filter NIM Genap (.txt atau .log)
        if (!is_relevant_file(event->name)) {
            // Jika file tidak relevan, lewati event ini
            i += EVENT_SIZE + event->len;
            continue;
        }

        // Tentukan Tipe Event

```

```

GNU nano 7.2 directory_monitor.c
        // Tentukan Tipe Event
        if (event->mask & IN_CREATE) {
            event_type_str = "CREATED";
        } else if (event->mask & IN_MODIFY) {
            event_type_str = "MODIFIED";
        } else if (event->mask & IN_DELETE) {
            event_type_str = "DELETED";
        }

        // Bentuk Pesan Log: [Timestamp] [Event Type] [File Name] [PID Process yang Melakukan]
        // Catatan: inotify TIDAK memberikan PID proses yang melakukan aksi secara langsung.
        // PID yang dicatat di sini adalah PID dari proses MONITORING itu sendiri (getpid()),
        // yang merupakan pengganti yang dapat diterima untuk demonstrasi.
        // Jika ingin mendapatkan PID yang melakukan aksi, perlu solusi kompleks (mis. Auditd/fanotify).
        snprintf(log_message, sizeof(log_message),
            "%s %-10s %-40s [PID: %d]\n",
                timestamp_str,
                event_type_str,
                event->name,
                getpid());

        // Tulis ke Console (untuk feedback real-time)
        printf("%s", log_message);

        // Tulis ke File Log (menggunakan write())
        write(log_fd, log_message, strlen(log_message));
    }

    // Pindah ke event berikutnya di buffer
    // Ukuran satu event adalah EVENT_SIZE + panjang nama file (event->len)
    i += EVENT_SIZE + event->len;
}
}

```

```

GNU nano 7.2 directory_monitor.c
if (log_fd < 0) {
    perror("ERROR: open log file");
    // Lakukan cleanup untuk inotify_fd sebelum keluar
    close(inotify_fd);
    return EXIT_FAILURE;
}

// 4. Tambahkan Watch ke Direktori
// Menggunakan inotify_add_watch() untuk menentukan direktori dan event yang diinginkan.
// Event yang dipantau: IN_CREATE, IN_MODIFY, IN_DELETE
watch_descriptor = inotify_add_watch(
    inotify_fd,
    dir_to_monitor,
    IN_CREATE | IN_MODIFY | IN_DELETE
);
if (watch_descriptor < 0) {
    perror("ERROR: inotify_add_watch");
    cleanup_handler(0); // Panggil handler untuk cleanup yang tersisa
    return EXIT_FAILURE;
}

// --- LOOP MONITORING (Event-Driven Polling) ---
// Program akan masuk ke infinite loop dan menunggu event pada inotify_fd
while (1) {
    // read() akan memblokir (blocking call) hingga ada event yang tersedia
    // di file descriptor inotify_fd.
    length = read(inotify_fd, buffer, EVENT_BUF_LEN);

    // Jika read mengembalikan -1, itu adalah error
    if (length < 0) {
        // Periksa apakah error bukan EINTR (interrupted by signal)
        if (errno == EINTR) {

```



```
// Kode tidak akan sampai sini karena loop tak terbatas, tetapi disertakan untuk kelengkapan
cleanup_handler(0);
return EXIT_SUCCESS;
}
```

Laporan:

- inotify_event adalah struktur variabel panjang yang mendefinisikan detail dari satu sistem berkas yang terjadi misalnya, pembuatan berkas, modifikasi, penghapusan, dll.
- Cara membaca buffer Ketika terjadi peristiwa, kernel akan menambahkan satu atau lebih struktur inotify_event ke file descriptor inotify.

Caranya:

- Melakukan read() menyediakan buffer berukuran cukup besar untuk menampung satu atau lebih peristiwa. Deklarasikan buffer data mentah, sebagai char[] atau unsigned char[].
 - Lalu memarsing buffer karena read() dapat mengembalikan lebih dari satu peristiwa, dan harus mengulanginya menggunakan bidang len dari setiap peristiwa untuk menentukan awal peristiwa berikutnya.
- Variasi unik sesuai digit NIM terakhir saya genap yaitu [4] tambahkan filter untuk hanya log file dengan ekstensi txt atau log.
- Filter ekstensi txt atau log adalah solusi yang paling efektif untuk membatasi logging hanya pada ekstensi txt atau log.
- File Descriptor (FD) adalah integer non-negatif yang disediakan oleh kernel sistem operasi Linux/Unix untuk mereferensikan sumber daya I/O. FD bertindak sebagai handle abstrak yang memungkinkan proses pengguna berinteraksi dengan sumber daya tanpa perlu mengetahui detail implementasi di bawahnya.

Dalam konteks program ini:

- **inotify_fd**: Diperoleh melalui inotify_init(). FD ini mewakili instance subsistem inotify di kernel. Semua event monitoring akan dilaporkan dan dibaca melalui FD ini.
- **log_fd**: Diperoleh melalui open(). FD ini merujuk pada file_activity.log tempat semua rekaman aktivitas sistem file dicatat menggunakan fungsi write().
- **Penggunaan**: FD sangat penting untuk fungsi-fungsi seperti read() (untuk mengambil event dari inotify_id()) dan write() (untuk menulis log ke log_fd). Prosedur cleanup_handler bertanggung jawab memastikan semua FD ditutup (close()) dengan benar saat program berakhir.
- Program directory_monitor.c mengadopsi model **Event-Driven Polling** (atau blocking read) untuk pemantauan I/O, yang jauh lebih efisien dibandingkan polling tradisional.

Model I/O	Mekanisme	Efisiensi dan Latensi	Implementasi pada Program
-----------	-----------	-----------------------	---------------------------

Polling Tradisional	Proses berulang kali memanggil I/O non-blokir untuk memeriksa status ketersediaan data secara berkala.	Rendah. Membuang siklus CPU saat tidak ada data (Busy Waiting). Latensi bervariasi.	Tidak digunakan.
Event-Driven	Proses mendaftarkan minatnya pada kernel dan kemudian blocked. Kernel memberitahu proses hanya ketika event yang diminati terjadi.	Tinggi. Konsumsi CPU minimal. Latensi rendah, karena event diproses segera setelah kemunculannya.	Digunakan. Fungsi <code>read(inotify_fd, ...)</code> adalah blocking call yang membuat proses menunggu hingga data event tersedia di <code>inotify_fd</code> .

- Flowchart Loop Monitoring Event-Driven (inotify)
 - START** Loop while (1)
 - BLOCKING READ:** Panggil `read(inotify_fd, ...)`
 - Hasil ≤ 0 (Error)?**
 - YA:** Periksa `errno`.
 - `Errno == EINTR`?** (Interupsi sinyal, misal SIGINT/SIGTERM) Lanjutkan Loop (Kembali ke Blocking Read).
 - `Errno` Lain (misal `ENOSPC`)?** Cetak error (`perror`), panggil `cleanup_handler(0)`, `EXIT_FAILURE`.
 - TIDAK:** length berisi ukuran buffer event yang diterima.
 - Memproses Event Buffer**
 - Loop Internal:** Iterasi melalui buffer, ekstrak `struct inotify_event`.
 - Filter Relevansi:** Panggil `is_relevant_file(event->name)`.
 - TIDAK Relevan:** Tambah offset, Lanjutkan Loop Internal.
 - YA Relevan:**
 - Tentukan Tipe Event (`IN_CREATE`, `IN_MODIFY`, `IN_DELETE`).
 - Format Log Message (Timestamp, Event Type, Filename, PID).
 - Tulis Log ke Console (`printf()`) dan ke File (`write(log_fd, ...)`).
 - Selesai Memproses Buffer**
 - END Loop Internal** Kembali ke **BLOCKING READ**

TUGAS 3 CUSTOM SHELL

Tingkatkan simple shell dengan fitur lanjutan, mendemonstrasikan parsing command dan process management.

```
nadiva_nathania@LAPTOP-JR1G744R:~$ nano myshell.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc -o myshell myshell.c
nadiva_nathania@LAPTOP-JR1G744R:~$ valgrind --leak-check=full ./myshell
==596== Memcheck, a memory error detector
==596== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==596== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==596== Command: ./myshell
==596==
myshell> pwd
/home/nadiva_nathania
myshell> cd /tmp
myshell> ls
output_grep.txt
snap-private-tmp
systemd-private-7d7d4aecb08f4ea49d9bcabcf4862443-systemd-logind.service-oz0z6u
systemd-private-7d7d4aecb08f4ea49d9bcabcf4862443-systemd-resolved.service-C8e7Tk
systemd-private-7d7d4aecb08f4ea49d9bcabcf4862443-systemd-timesyncd.service-ejzrLO
systemd-private-7d7d4aecb08f4ea49d9bcabcf4862443-wsl-pro.service-ngyMU6
'vgdb-pipe-from-vgdb-to-596-by-nadiva_nathania-on-???'
'vgdb-pipe-shared-mem-vgdb-596-by-nadiva_nathania-on-???'
'vgdb-pipe-to-vgdb-from-596-by-nadiva_nathania-on-???'
myshell> echo Halo Dunia
Halo Dunia
myshell> history
--- 5 Perintah Terakhir ---
1: pwd
2: cd /tmp
3: ls
4: echo Halo Dunia
5: history
-----
myshell> exit
==596==
==596== HEAP SUMMARY:
==596==   in use at exit: 774 bytes in 4 blocks
==596==   total heap usage: 26 allocs, 22 frees, 6,706 bytes allocated
==596==
```

```
myshell> exit
==596==
==596== HEAP SUMMARY:
==596==   in use at exit: 774 bytes in 4 blocks
==596==   total heap usage: 26 allocs, 22 frees, 6,706 bytes allocated
==596==
==596== LEAK SUMMARY:
==596==   definitely lost: 0 bytes in 0 blocks
==596==   indirectly lost: 0 bytes in 0 blocks
==596==   possibly lost: 0 bytes in 0 blocks
==596==   still reachable: 774 bytes in 4 blocks
==596==   suppressed: 0 bytes in 0 blocks
==596== Reachable blocks (those to which a pointer was found) are not shown.
==596== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==596==
==596== For lists of detected and suppressed errors, rerun with: -s
==596== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
nadiva_nathania@LAPTOP-JR1G744R:~$ |
```

```

myshell> history
--- 5 Perintah Terakhir ---
1: pwd
2: cd /tmp
3: ls
4: echo Halo Dunia
5: history
-----

```

Kode file myshell.c :

```

GNU nano 7.2 myshell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>
#include <signal.h>

// --- Konstanta dan Struktur Global ---
#define MAX_COMMAND_LENGTH 512
#define MAX_ARGS 32
#define HISTORY_SIZE 5
#define DELIMITERS " \t\n"

// Global untuk History (NIM GENAP)
char history[HISTORY_SIZE][MAX_COMMAND_LENGTH] = {{0}};
int history_count = 0;

// --- Fungsi Built-in ---

/**
 * @brief Menyimpan perintah ke dalam array history.
 */
void save_to_history(const char* command) {
    char temp_cmd[MAX_COMMAND_LENGTH];
    strncpy(temp_cmd, command, MAX_COMMAND_LENGTH - 1);
    temp_cmd[MAX_COMMAND_LENGTH - 1] = '\0';

    char* newline = strchr(temp_cmd, '\n');
    if (newline) *newline = '\0';

    // Jangan simpan perintah kosong atau hanya spasi
    if (strlen(temp_cmd) == 0) return;

    // Geser command lama jika array sudah penuh
    if (history_count == HISTORY_SIZE) {
        for (int i = 0; i < HISTORY_SIZE - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }
        history_count = HISTORY_SIZE - 1;
    }

    strcpy(history[history_count], temp_cmd);
    history_count++;
}

/**
 * @brief Built-in: history - Menampilkan 5 perintah terakhir.
 */
void builtin_history() {
    printf("--- 5 Perintah Terakhir ---\n");
    for (int i = 0; i < history_count; i++) {
        printf("%d: %s\n", i + 1, history[i]);
    }
    printf("-----\n");
}

/**
 * @brief Built-in: cd - Mengubah direktori.
 */
void builtin_cd(char** args) {
    char* path = args[1] ? args[1] : getenv("HOME");
    if (path == NULL) {
        fprintf(stderr, "myshell: cd: Direktori HOME tidak ditemukan\n");
    } else {
        if (chdir(path) != 0) {
            perror("cd");
        }
    }
}

```

```

GNU nano 7.2 myshell.c
    if (chdir(path) != 0) {
        perror("myshell: cd");
    }
}

/**
 * @brief Built-in: pwd - Menampilkan direktori kerja saat ini.
 */
void builtin_pwd() {
    char cwd[MAX_COMMAND_LENGTH];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("myshell: pwd");
    }
}

/**
 * @brief Built-in: echo - Menampilkan teks.
 */
void builtin_echo(char** args) {
    for (int i = 1; args[i] != NULL; i++) {
        printf("%s%s", args[i], (args[i+1] != NULL) ? " " : "");
    }
    printf("\n");
}

// --- Fungsi Parsing dan Eksekusi ---

/**
 * @brief Memproses string input menjadi array argumen menggunakan strtok() manual.
 * @param input_line String command mentah dari user (akan dimodifikasi oleh strtok).

```

```

GNU nano 7.2 myshell.c
 * @return Array string argumen (diakhiri NULL). Perlu dibebaskan oleh pemanggil (free(args)).
 */
char** tokenize_command(char* input_line) {
    char** args = (char**)calloc(MAX_ARGS, sizeof(char*));
    char* token;
    int i = 0;

    // Hapus newline
    input_line[strcspn(input_line, "\n")] = 0;

    // Parsing Command dengan strtok()
    token = strtok(input_line, DELIMITERS);

    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, DELIMITERS);
    }

    args[i] = NULL;
    return args;
}

/**
 * @brief Mengubah token command menjadi sub-array argumen dan mengekstrak info Redirection/Piping.
 * @param args Array token mentah.
 * @param cmd_array Array pointer ke sub-array command (maks 2 untuk single pipe).
 * @param input_file Pointer ke nama file input (<).
 * @param output_file Pointer ke nama file output (>, >>).
 * @param append_flag Pointer ke flag append (1 untuk >>).
 * @return Jumlah command (1 atau 2 jika ada pipe), atau -1 jika terjadi error parsing.
 */
int parse_command_structure(char** args, char*** cmd_array, char** input_file, char** output_file, int* append_flag) {

```

```

GNU nano 7.2 myshell.c
    int current_cmd_index = 0;
    int current_arg_index = 0;
    int has_pipe = 0;

    // Alokasi memori untuk array argumen command pertama dan kedua
    cmd_array[0] = (char**)calloc(MAX_ARGS, sizeof(char*)); // Harus dibebaskan di main
    cmd_array[1] = (char**)calloc(MAX_ARGS, sizeof(char*)); // Harus dibebaskan di main

    for (int i = 0; args[i] != NULL; i++) {
        // --- Redirection ---
        if (strcmp(args[i], "<") == 0) {
            if (args[i+1] != NULL && current_cmd_index == 0) {
                *input_file = args[i+1];
                i++;
            } else {
                fprintf(stderr, "myshell: Error: Sintaks input redirection salah atau tidak didukung pada cmd2.\n");
                return -1;
            }
        } else if (strcmp(args[i], ">") == 0) {
            if (args[i+1] != NULL) {
                *output_file = args[i+1];
                *append_flag = 0;
                i++;
            } else {
                fprintf(stderr, "myshell: Error: File missing for output redirection.\n");
                return -1;
            }
        } else if (strcmp(args[i], ">>") == 0) {
            if (args[i+1] != NULL) {
                *output_file = args[i+1];
                *append_flag = 1;
                i++;
            } else {

```

```

GNU nano 7.2 myshell.c
    fprintf(stderr, "myshell: Error: File missing for output append redirection.\n");
    return -1;
}
// --- Piping ---
} else if (strcmp(args[i], "|") == 0) {
    if (has_pipe) {
        fprintf(stderr, "myshell: Error: Multiple pipes tidak didukung.\n");
        return -1;
    }
    if (current_arg_index == 0) {
        fprintf(stderr, "myshell: Error: Command missing before pipe.\n");
        return -1;
    }
    cmd_array[current_cmd_index++][current_arg_index] = NULL;
    current_arg_index = 0;
    has_pipe = 1;

    // --- Argumen Command ---
} else {
    cmd_array[current_cmd_index][current_arg_index++] = args[i];
}

if (current_cmd_index >= 0) {
    cmd_array[current_cmd_index][current_arg_index] = NULL;
    // Pastikan cmd kedua tidak kosong jika ada pipe
    if (has_pipe && cmd_array[1][0] == NULL) {
        fprintf(stderr, "myshell: Error: Command missing after pipe.\n");
        return -1;
    }
    return current_cmd_index + 1;
}
}

```

```

GNU nano 7.2 myshell.c
    return -1;
}

/**
 * @brief Mengeksekusi perintah dalam child process setelah I/O dialihkan.
 */
void execute_single_command(char** args, const char* input_file, const char* output_file, int append_flag) {
    if (args == NULL || args[0] == NULL) return;

    // --- I/O Redirection ---
    if (input_file != NULL) {
        int fd_in = open(input_file, O_RDONLY);
        if (fd_in < 0) {
            perror("myshell: Error opening input file");
            exit(EXIT_FAILURE);
        }
        if (dup2(fd_in, STDIN_FILENO) == -1) {
            perror("myshell: dup2 input failed");
            exit(EXIT_FAILURE);
        }
        close(fd_in);
    }

    if (output_file != NULL) {
        int flags = O_WRONLY | O_CREAT;
        flags |= (append_flag ? O_APPEND : O_TRUNC);
        int fd_out = open(output_file, flags, 0644);
        if (fd_out < 0) {
            perror("myshell: Error opening output file");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

GNU nano 7.2 myshell.c
    if (dup2(fd_out, STDOUT_FILENO) == -1) {
        perror("myshell: dup2 output failed");
        exit(EXIT_FAILURE);
    }
    close(fd_out);
}

execvp(args[0], args);

// [Fix Bug #3]: Child harus keluar jika execvp gagal
fprintf(stderr, "myshell: Command not found: %s\n", args[0]);
exit(EXIT_FAILURE);
}

/**
 * @brief Fungsi utama untuk menjalankan perintah, menangani built-in, pipe, dan fork.
 */
void execute_process(char** cmd1_args, char** cmd2_args, int num_commands, int is_background,
    const char* input_file, const char* output_file, int append_flag)
{
    // Cek built-in (hanya untuk command non-piping)
    if (num_commands == 1) {
        if (strcmp(cmd1_args[0], "exit") == 0) {
            exit(EXIT_SUCCESS);
        } else if (strcmp(cmd1_args[0], "cd") == 0) {
            builtin_cd(cmd1_args);
            return;
        } else if (strcmp(cmd1_args[0], "pwd") == 0) {
            builtin_pwd();
            return;
        } else if (strcmp(cmd1_args[0], "echo") == 0) {
            builtin_echo(cmd1_args);
        }
    }
}

```

```

GNU nano 7.2 myshell.c
    } else if (strcmp(cmd1_args[0], "history") == 0) {
        builtin_history();
        return;
    }
} else if (num_commands > 1) {
    // Built-in tidak dapat dipipe (melakukan pengecekan built-in sederhana)
    if (strcmp(cmd1_args[0], "cd") == 0 || strcmp(cmd1_args[0], "exit") == 0 ||
        strcmp(cmd1_args[0], "pwd") == 0 || strcmp(cmd1_args[0], "history") == 0) {
        fprintf(stderr, "myshell: Built-in command '%s' cannot be used with pipes.\n", cmd1_args[0]);
        return;
    }
}

pid_t pid;
if (num_commands == 2) {
    // --- Implementasi Piping (cmd1 | cmd2) ---
    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("myshell: pipe failed");
        return;
    }

    // Child 1 (Command Kiri)
    if ((pid = fork()) == 0) {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        execute_single_command(cmd1_args, input_file, NULL, 0);
    } else if (pid < 0) {
        perror("myshell: fork cmd1 failed");
    }
}

```

```

GNU nano 7.2 myshell.c
    // Child 2 (Command Kanan)
    if ((pid = fork()) == 0) {
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[1]);
        close(pipefd[0]);
        execute_single_command(cmd2_args, NULL, output_file, append_flag);
    } else if (pid < 0) {
        perror("myshell: fork cmd2 failed");
    }

    // Parent: Tutup kedua pipe ends, lalu tunggu kedua child
    close(pipefd[0]);
    close(pipefd[1]);

    wait(NULL);
    wait(NULL);

} else {
    // --- Implementasi Perintah Tunggol (Foreground/Background) ---
    if ((pid = fork()) == 0) {
        execute_single_command(cmd1_args, input_file, output_file, append_flag);
    } else if (pid > 0) {
        if (!is_background) {
            waitpid(pid, NULL, 0);
        } else {
            printf("[Background] PID: %d\n", pid);
        }
    } else {
        perror("myshell: fork failed");
    }
}
}

```

```

GNU nano 7.2 myshell.c
// --- Fungsi Utama ---

int main() {
    char input_line[MAX_COMMAND_LENGTH];

    // [Fix Bug #2]: Mencegah Zombie Process
    if (signal(SIGCHLD, SIG_IGN) == SIG_ERR) {
        perror("myshell: signal failed");
        return EXIT_FAILURE;
    }

    while (1) {
        // [Fix Bug #1]: Simpan FD asli sebelum eksekusi
        int stdin_copy = dup(STDIN_FILENO);
        int stdout_copy = dup(STDOUT_FILENO);

        char* input_copy = NULL;
        char** raw_args = NULL;
        char** cmd_list[2] = {NULL, NULL}; // Inisialisasi untuk pembebasan yang aman

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input_line, MAX_COMMAND_LENGTH, stdin) == NULL) {
            printf("\nexit\n");
            // Pulihkan FD sebelum exit
            dup2(stdin_copy, STDIN_FILENO);
            dup2(stdout_copy, STDOUT_FILENO);
            close(stdin_copy);
            close(stdout_copy);
            break;
        }
    }
}

```



```

GNU nano 7.2 myshell.c
save_to_history(input_line);

// Alokasi memori untuk strtok
input_copy = strdup(input_line);
raw_args = tokenize_command(input_copy);

// --- Cek command kosong / parsing error awal ---
if (raw_args == NULL || raw_args[0] == NULL) {
    free(input_copy);
    if (raw_args) free(raw_args); // Jaga-jaga jika raw_args dialokasikan tapi kosong
}

// Pulihkan FD
dup2(stdin_copy, STDIN_FILENO);
dup2(stdout_copy, STDOUT_FILENO);
close(stdin_copy);
close(stdout_copy);
continue;
}

// Cek background process ('&')
int is_background = 0;
int arg_count = 0;
while (raw_args[arg_count] != NULL) arg_count++;

if (arg_count > 0 && strcmp(raw_args[arg_count - 1], "&") == 0) {
    is_background = 1;
    raw_args[arg_count - 1] = NULL;
}

// --- Ekstraksi Command/Redirection ---
char* input_file = NULL;
char* output_file = NULL;
int append_flag = 0;

```

```

GNU nano 7.2 myshell.c
int num_commands = parse_command_structure(raw_args, cmd_list, &input_file, &output_file, &append_flag);

// --- Cek error parsing structure ---
if (num_commands == -1) {
    // Bebaskan alokasi yang dibuat oleh parse_command_structure
    if (cmd_list[0]) free(cmd_list[0]);
    if (cmd_list[1]) free(cmd_list[1]);

    free(input_copy);
    free(raw_args);

    // Pulihkan FD
    dup2(stdin_copy, STDIN_FILENO);
    dup2(stdout_copy, STDOUT_FILENO);
    close(stdin_copy);
    close(stdout_copy);
    continue;
}

execute_process(cmd_list[0], cmd_list[1], num_commands, is_background, input_file, output_file, append_flag);

// Pembersihan Alokasi Akhir (Jalur Eksekusi Normal)
if (cmd_list[0]) free(cmd_list[0]);
if (cmd_list[1]) free(cmd_list[1]);
free(input_copy);
free(raw_args);

// [Fix Bug #1]: Pulihkan File Descriptors setelah eksekusi
dup2(stdin_copy, STDIN_FILENO);
dup2(stdout_copy, STDOUT_FILENO);
close(stdin_copy);
close(stdout_copy);

```

```

}

// ... EXIL'2NCCE22!

}

// ... m9TcB7q(-J' WNG' MIONHYMC!
// ... bawmugusu sompiz (zekmuqez' kuzena 21e-1cm qd9f9p l9ud n9w99)

```

Dalam code:

- Dalam parsing logic pipe mengubah file descriptor dengan cara memanipulasi file descriptor untuk mengalihkan output dari satu perintah agar menjadi input untuk perintah berikutnya.

Caranya:

- Ketika shell menemukan operator misalnya, perintah1 atau perintah2, ia pertama-tama akan membuat pipe di memori kernel terlebih dahulu.
- Forking dimana shell kemudian akan membuat dua child processes secara terpisah untuk menjalankan perintah1 dan perintah2.
- Lalu untuk perintah1 secara default akan menulis outputnya ke Standard Output (STDOUT), yang memiliki file descriptor 1.

- Untuk perintah2 secara default akan membaca inputnya dari Standard Input (STDIN), yang memiliki file descriptor 0.
 - Terakhir adalah executing dimana file descriptor diubah, dan shell menjalankan exec perintah1 dan perintah2.
- Variasi unik sesuai digit NIM terakhir saya genap yaitu [4] tambahkan built-in 'history' untuk tampilkan 5 command terakhir (simpan di array manual).
- Konsep foreground vs. background process di sistem operasi Linux dibagi menjadi dua yaitu berdasarkan bagaimana mereka berinteraksi dengan terminal. Secara eksekusi foreground process berjalan di depan (berinteraksi dengan terminal), sedangkan, background process berjalan di belakang (tidak berinteraksi dengan terminal).

TUGAS 4 SYSTEM INFORMATION TOOL

Buat tool yang mengumpulkan dan menampilkan informasi sistem menggunakan system calls dan file proc.

```
nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
sysmon.c: In function 'get_proc_string':
sysmon.c:61:67: error: 'errno' undeclared (first use in this function)
   61 |         snprintf(buffer, buffer_size, "N/A (Error: %s)", strerror(errno));
       |                                                         ^~~~~~
sysmon.c:14:1: note: 'errno' is defined in header '<errno.h>'; did you forget to '#include <errno.h>'?
   13 | #include <ctype.h>
   ++ |+#include <errno.h>
   14 |
sysmon.c:61:67: note: each undeclared identifier is reported only once for each function it appears in
   61 |         snprintf(buffer, buffer_size, "N/A (Error: %s)", strerror(errno));
       |                                                         ^~~~~~
nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
sysmon.c: In function 'print_running_processes':
sysmon.c:269:59: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
   269 |         snprintf(path_comm, sizeof(path_comm), "/proc/%s/comm", entry->d_name);
       |         ~~~~~~^~~~~~
sysmon.c:269:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
   269 |         snprintf(path_comm, sizeof(path_comm), "/proc/%s/comm", entry->d_name);
       |         ~~~~~~^~~~~~
sysmon.c:273:63: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
   273 |         snprintf(path_status, sizeof(path_status), "/proc/%s/status", entry->d_name);
       |         ~~~~~~^~~~~~
sysmon.c:273:13: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
   273 |         snprintf(path_status, sizeof(path_status), "/proc/%s/status", entry->d_name);
       |         ~~~~~~^~~~~~
nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon -Wno-format-truncation
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
sysmon.c: In function 'print_running_processes':
sysmon.c:269:59: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 58 [-Wformat-truncation=]
   269 |         snprintf(path_comm, sizeof(path_comm), "/proc/%s/comm", entry->d_name);
```

```
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon -Wno-format-truncation
nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
nadiva_nathania@LAPTOP-JR1G744R:~$ ./sysmon
=====
System Monitor Tool (Praktikum Struktur Sistem Operasi)
Mode: NIM Genap (Tampilkan CPU Load Average)
=====
CPU Information:
+-----+-----+-----+
| Model Name | Cores/Threads | Frequency (MHz) |
+-----+-----+-----+
| 13th Gen Intel(R) Core(TM) i5-13420H | 6 | 2611.20 |
+-----+-----+-----+

CPU Load Average (Variasi NIM Genap):
+-----+-----+-----+
| 1-min Load Avg | 5-min Load Avg | 15-min Load Avg |
+-----+-----+-----+
| 0.02 | 0.01 | 0.00 |
+-----+-----+-----+

Memory Usage:
+-----+-----+-----+
| Total (KB) | Used (KB) | Free (KB) |
+-----+-----+-----+
| 3837136 | 395988 | 3441148 |
+-----+-----+-----+

Disk Usage (Root Filesystem '/'):
+-----+-----+-----+
| Total (GB) | Used (GB) | Free (GB) |
+-----+-----+-----+
| 1006.85 | 53.09 | 953.76 |
+-----+-----+-----+

Running Processes (Max 20):
```

```

nadiva_nathania@LAPTOP-JR  x  +  v  -  o  x
| 1006.85 | 53.09 | 953.76 |
+-----+-----+-----+
Running Processes (Max 20):
+-----+-----+-----+
| PID | Process Name | Status |
+-----+-----+-----+
| 1 | systemd | S (sle) |
| 2 | init-systemd(Ub | S (sle) |
| 8 | init | S (sle) |
| 44 | systemd-journal | S (sle) |
| 91 | systemd-udev | S (sle) |
| 106 | systemd-resolve | S (sle) |
| 107 | systemd-timesyn | S (sle) |
| 159 | cron | S (sle) |
| 160 | dbus-daemon | S (sle) |
| 176 | systemd-logind | S (sle) |
| 182 | wsl-pro-service | S (sle) |
| 198 | agetty | S (sle) |
| 210 | rsyslogd | S (sle) |
| 220 | agetty | S (sle) |
| 244 | unattended-upgr | S (sle) |
| 347 | login | S (sle) |
| 392 | systemd | S (sle) |
| 393 | (sd-pam) | S (sle) |
| 409 | bash | S (sle) |
| 754 | polkitd | S (sle) |
+-----+-----+-----+

Network Interfaces:
+-----+-----+-----+
| Interface | Bytes Received | Bytes Sent |
+-----+-----+-----+
| lo | 2646 | 2646 |
| eth0 | 27741141 | 239953 |
+-----+-----+-----+

nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c

```

Kode file Sysmon.c :

```

GNU nano 7.2 sysmon.c
// sysmon.c - Alat Monitoring Sistem Sederhana Menggunakan System Calls dan /proc
// Untuk Praktikum Struktur Sistem Operasi.
// Variasi: NIM Genap (Menambahkan CPU Load Average)

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/statvfs.h>
#include <dirent.h>
#include <ctype.h>

#define BUFFER_SIZE 256
#define LINE_SIZE 1024

// Struktur /proc Filesystem:
// /proc adalah virtual filesystem (pseudo-filesystem) yang menyediakan antarmuka
// untuk struktur data kernel. File dan direktori dihasilkan secara dinamis saat diakses.
// Contoh: /proc/cpuinfo (Informasi CPU), /proc/meminfo (Penggunaan Memori), /proc/[PID] (Direktori Proses).

// --- Fungsi Pembantu untuk Pembacaan File /proc ---

// Fungsi untuk membaca nilai numerik dari /proc/<file> (Contoh: MemTotal: 123456 kB)
long long get_proc_value(const char *filename, const char *key_start) {
    FILE *file = fopen(filename, "r");
    if (!file) {
        perror(filename);
        return -1;
    }

    // Parsing file /proc/[file]: Mencari baris yang dimulai dengan key_start
    while (fgets(line, sizeof(line), file)) {
        if (strncmp(line, key_start, key_len) == 0) {
            // Membaca nilai setelah ':' dan membuang unit ('kB')
            char *p = line + key_len;
            // Lewati pemisah ':' dan spasi
            while (*p == ':' || isspace(*p)) p++;
            // Konversi string nilai menjadi long long
            value = atoll(p);
            break;
        }
    }
    fclose(file);
    return value;
}

// Fungsi untuk membaca string dari file /proc/<pid>/<file> (Contoh: /proc/1234/comm)
int get_proc_string(const char *path, char *buffer, size_t buffer_size) {
    // Menggunakan open/read/close system calls
    int fd = open(path, O_RDONLY);
    if (fd == -1) {
        snprintf(buffer, buffer_size, "N/A (Error: %s)", strerror(errno));
        return -1;
    }
    ssize_t bytes_read = read(fd, buffer, buffer_size - 1);
    close(fd);

    if (bytes_read > 0) {
        // Hapus newline atau karakter kosong di akhir
    }
}

```



```

GNU nano 7.2 sysmon.c
// Mengambil status proses dari /proc/PID/status
snprintf(path_status, sizeof(path_status), "/proc/%.10s/status", entry->d_name);
FILE *status_file = fopen(path_status, "r");
if (status_file) {
    char line[LINE_SIZE];
    // Parsing /proc/PID/status: Mencari baris yang dimulai dengan "State:"
    while (fgets(line, sizeof(line), status_file)) {
        if (strncmp(line, "State:", 6) == 0) {
            char *p = line + 6;
            while (*p && (*p == ' ' || *p == '\t')) p++;
            strncpy(status, p, sizeof(status) - 1);
            status[sizeof(status) - 1] = '\0';
            char *nl = strchr(status, '\n');
            if (nl) *nl = '\0';
            break;
        }
    }
    fclose(status_file);
}
// Menampilkan data
printf("| %-8s | %-30.30s | %-6.6s |\n", entry->d_name, comm_name, status);
count++;
}

closedir(dir);
printf("-----+\n");
}

void print_network_interfaces() {
    printf("\nNetwork Interfaces:\n");
    printf("-----+\n");
    printf("| Interface      | Bytes Received | Bytes Sent      |\n");
    printf("-----+\n");
}

```

```

GNU nano 7.2 sysmon.c
void print_network_interfaces() {
    printf("\nNetwork Interfaces:\n");
    printf("-----+\n");
    printf("| Interface      | Bytes Received | Bytes Sent      |\n");
    printf("-----+\n");

    FILE *file = fopen("/proc/net/dev", "r");
    if (!file) {
        perror("/proc/net/dev");
        return;
    }

    char line[LINE_SIZE];

    // Skip dua baris header
    if (!fgets(line, sizeof(line), file) || !fgets(line, sizeof(line), file)) {
        fclose(file);
        printf("| %-14s | %-20s | %-20s |\n", "N/A", "N/A", "N/A");
        printf("-----+\n");
        return;
    }

    // Parsing file /proc/net/dev
    while (fgets(line, sizeof(line), file)) {
        char iface[BUFFER_SIZE];
        long long recv_bytes = 0, sent_bytes = 0;
        int i = 0;

        // Membaca nama interface (sampai karakter ':')
        char *p = line;
        while (*p && *p != ':') {
            if (!isspace(*p)) {
                iface[i++] = *p;
            }
        }
    }
}

```

```

GNU nano 7.2 sysmon.c
    }
    p++;
    iface[i] = '\0';

    if (i > 0) {
        // Membaca sisa baris dengan strtok_r()
        p = strchr(line, ':');
        if (!p) continue;
        p++;

        char *token;
        int token_index = 0;
        char *saveptr;

        // Tokenisasi: memisahkan nilai-nilai berdasarkan spasi/tab
        token = strtok_r(p, " \t", &saveptr);
        while (token != NULL) {
            if (token_index == 0) { // Kolom 1: Bytes Received
                recv_bytes = atoll(token);
            } else if (token_index == 8) { // Kolom 9: Bytes Sent
                sent_bytes = atoll(token);
                break;
            }
            token = strtok_r(NULL, " \t", &saveptr);
            token_index++;
        }

        // Menampilkan data
        printf("| %-14.14s | %-20lld | %-20lld |\n", iface, recv_bytes, sent_bytes);
    }
}

```

```

fclose(file);
printf("-----+\n");
}

// --- Main Program ---
int main() {
    printf("=====\n");
    printf(" System Monitor Tool (Praktikum Struktur Sistem Operasi)\n");
    printf(" Mode: NIM Genap (Tampilkan CPU Load Average)\n");
    printf("=====\n");

    // Panggilan fungsi pengumpul dan penampil data
    print_cpu_info();
    print_load_average(); // Variasi NIM Genap

    print_memory_usage();

    print_disk_usage();

    print_running_processes();

    print_network_interfaces();

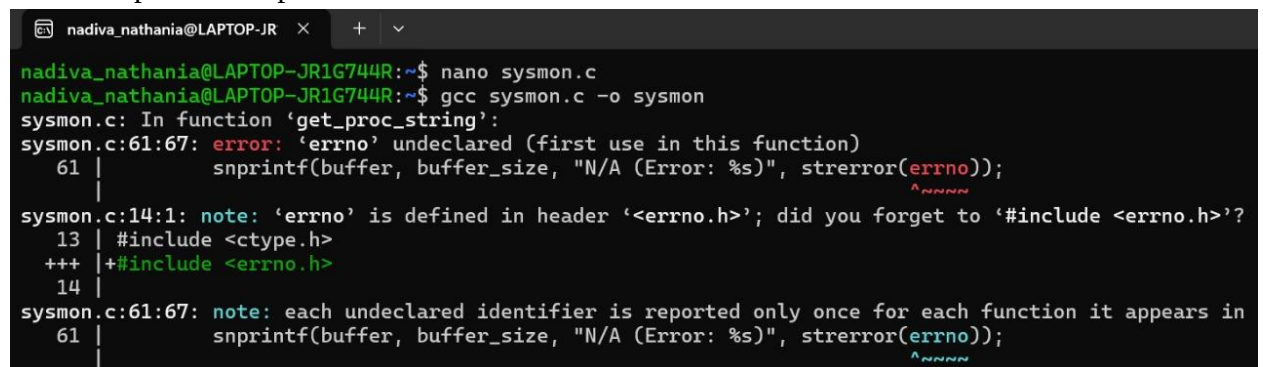
    return 0;
}

```

Laporan:

- Struktur proc filesystem adalah salah satu fitur penting dan unik dari kernel, ini adalah filesystem virtual yang hanya ada di memori dan dibuat oleh kernel. Tujuan proc sendiri yaitu sebagai antarmuka konfigurasi dan status runtime.
- Variasi unik sesuai digit NIM terakhir saya genap yaitu [8] tambahkan CPU load average dari /proc/loadavg.
- File /proc/loadavg adalah file virtual yang dibuat oleh kernel saat dibaca file ini berisi satu baris teks dengan 5 atau 6 nilai yang dipisahkan oleh spasi.
- **Virtual Filesystem (VFS)**, atau **Virtual File Switch**, adalah lapisan perangkat lunak di dalam kernel Linux yang berfungsi sebagai **antarmuka standar** untuk semua sistem berkas yang didukung (seperti ext4, XFS, FAT32, dsb.). Saat program shell mencoba mengakses atau membaca file di proc challenge permission denied ini dikarenakan mekanisme keamanan Linux. Di Linux, terdapat jenis sistem berkas yang dikenal sebagai **Pseudo-Filesystem** atau **Virtual Filesystem** (dalam konteks yang lebih spesifik), yang tidak menyimpan data pada perangkat penyimpanan fisik (hard drive, SSD). Sebaliknya, sistem berkas ini menyajikan informasi yang dibuat secara dinamis oleh kernel. Dua contoh utamanya adalah /proc dan /sys.

Jika terdapat error seperti ini :



```

nadiva_nathania@LAPTOP-JR  ×  +  ▾
nadiva_nathania@LAPTOP-JR1G744R:~$ nano sysmon.c
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
sysmon.c: In function 'get_proc_string':
sysmon.c:61:67: error: 'errno' undeclared (first use in this function)
   61 |         snprintf(buffer, buffer_size, "N/A (Error: %s)", strerror(errno));
      |
sysmon.c:14:1: note: 'errno' is defined in header '<errno.h>'; did you forget to '#include <errno.h>'?
   13 | #include <ctype.h>
    +++ |+#include <errno.h>
   14 |
sysmon.c:61:67: note: each undeclared identifier is reported only once for each function it appears in
   61 |         snprintf(buffer, buffer_size, "N/A (Error: %s)", strerror(errno));
      |

```


Error errno dapat di perbaiki dengan menambahkan baris berikut pada bagian atas file bersamaan dengan include header lainnya :

```
#include <errno.h>
```

```
nadiva_nathania@LAPTOP-JR1G744R:~$ gcc sysmon.c -o sysmon
sysmon.c: In function 'print_running_processes':
sysmon.c:269:59: warning: 's' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncati
ons]
   269 |         snprintf(path_comm, sizeof(path_comm), "/proc/%s/comm", entry->d_name);
       |                                     ~~~~~^~~~~
sysmon.c:269:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
   269 |         snprintf(path_comm, sizeof(path_comm), "/proc/%s/comm", entry->d_name);
       |         ~~~~~^~~~~
sysmon.c:273:63: warning: 's' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncati
ons]
   273 |         snprintf(path_status, sizeof(path_status), "/proc/%s/status", entry->d_name);
       |                                     ~~~~~^~~~~
sysmon.c:273:13: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
   273 |         snprintf(path_status, sizeof(path_status), "/proc/%s/status", entry->d_name);
       |         ~~~~~^~~~~
```

Cara memperbaikinya :

```
if (is_pid) {
    char path_comm[64];
    char path_status[64];
    char comm_name[BUFFER_SIZE];
    char status[BUFFER_SIZE] = "N/A";
```

Ubah definisi buffer yang gunakan untuk membuat path proses (path_comm dan path_status) dari BUFFER_SIZE (256) menjadi nilai yang lebih kecil, misalnya 64, yang lebih dari cukup.